Here is your complete Replit Agent brief:

---

# REPLIT AGENT BRIEF

## PathAble – AI-Powered Smart Campus Navigation for Differently Abled Students

---

## 1. APP OVERVIEW

**App Name:** PathAble

**One-line description:** An AI-powered, fully software-based campus navigation web app that gives differently abled students personalized, accessible, real-time route guidance, obstacle alerts, and emergency support — all from a single smartphone browser with zero hardware required.

**Core problem it solves:** Differently abled students on college campuses face daily navigation challenges — inaccessible routes, broken elevators, no advance warning of obstacles, and no way to get emergency help fast. Existing navigation apps like Google Maps are not accessibility-aware and do not know the interior of campus buildings. PathAble solves this with a personalized, AI-guided campus navigation system that adapts to each student's disability type and works entirely in software.

**Target users:** Primary — differently abled college students (mobility, visual, hearing, cognitive impairments). Secondary — campus administrators and faculty who manage accessibility infrastructure and accommodate students.

---

## 2. FULL FEATURE LIST

### Priority 1 — Must demo to judges

#### F01 — Disability Profile Setup and Personalization

- Multi-step onboarding flow where students select disability type (mobility/wheelchair, visual impairment, hearing impairment, cognitive/learning disability, or combination)
- Student selects mobility aid if applicable: manual wheelchair, electric wheelchair, crutches, white cane, none

- Preferred navigation output selected: voice only, haptic only, visual only, or all three combined
- Profile stored in localStorage and applied automatically on every return visit
- All personalization logic runs client-side — no backend or medical database needed
- Profile editable any time from the settings page in under 60 seconds

**F02 — Accessible Route Navigation with Campus Map**

- Interactive campus map rendered using Leaflet.js with a custom GeoJSON overlay of campus paths
- Every path segment in the GeoJSON is tagged with: surface type, slope level (flat/mild/steep), width category (narrow/standard/wide), and step count
- Wheelchair routing removes all edges tagged with stairs or steep slope from the routing graph before calculating path
- Route calculated using a shortest-path algorithm (Dijkstra's) implemented in JavaScript on the client side
- Route displayed as a highlighted polyline on the Leaflet map with step-by-step turn instructions in a panel below
- Three route options shown ranked by: shortest distance, fewest obstacles, and most rest stops

**F03 — AI Navigation Assistant (Claude API)**

- A chat-style assistant panel on the navigation screen powered by the Anthropic Claude API
- Student types or speaks (via browser Web Speech API) a natural language query: "How do I get to the library?", "Is there an accessible washroom near Block B?", "What do I do if I get stuck?"
- The app sends the query to Claude with a system prompt containing the full campus knowledge base: building names, room numbers, accessible facilities, office hours, and current obstacle reports
- Claude returns a plain-language answer which is displayed in the chat panel and read aloud via the browser's SpeechSynthesis API
- The assistant also auto-generates a step-by-step accessible response checklist whenever a crisis or obstacle is reported
- Full pipeline: browser mic → Web Speech API → Claude API → SpeechSynthesis → audio — no external hardware

**F04 — Real-Time Obstacle Reporting and Crowdsourced Alerts**

- Any logged-in user taps "Report an Issue" button, selects issue type from: Elevator Down, Wet Floor, Blocked Ramp, Construction, Missing Signage, Other
- User selects location from a dropdown of campus zones and buildings — no GPS hardware needed
- Report is saved to the database with timestamp, reporter name, zone, issue type, and description
- All active reports appear immediately as red warning pins on the shared Leaflet map visible to all users
- Reports expire automatically after 4 hours unless confirmed by another user

- If 3 or more users confirm the same report, the routing engine recalculates all active routes avoiding that zone and notifies affected users with a toast notification
- Admin can dismiss or escalate any report from the admin dashboard

**F05 — One-Tap SOS Emergency Alert**

- A large red SOS button permanently visible on the bottom navigation bar of the student app
- On tap, a confirmation modal appears with a 5-second countdown and "Cancel" button to prevent accidental triggers
- On confirmation: a push notification is sent to all admin accounts via the app's notification system, the student's zone and disability type are included in the alert, and a live incident timer starts on the admin dashboard
- An in-app emergency contacts list (set during profile setup) shows the student who has been alerted
- The SOS log is stored in the database for admin review and welfare follow-up
- The nearest first aid location and safe zone checkpoint are highlighted on the student's map immediately after SOS is triggered

**F06 — Admin Crisis Command Dashboard**

- A separate web dashboard route (`/admin`) accessible by admin accounts only
- Displays a live list of all active obstacle reports, SOS alerts, and system-wide notifications in a three-column layout
- Each incident card shows: reporter name, zone, issue type, time elapsed, severity badge (auto-assigned by AI), and action buttons: Confirm, Dismiss, Escalate
- Admins can push a campus-wide alert message (e.g. "East elevator under maintenance until 3 PM") that appears as a red banner on all student screens instantly via polling
- Key metrics displayed at the top: active incidents today, SOS events this week, most-reported zones, average resolution time
- Monthly report download button generates a structured text summary of all incidents

**F07 — Crowd Density Heatmap**

- A heatmap overlay on the campus Leaflet map showing predicted crowd density by zone
- Density is pre-calculated from a hardcoded timetable dataset (class schedules by building and hour) included as a JSON file in the project
- The current hour of day is read from the browser clock and the matching density data is applied to colour the map zones: green (clear), amber (moderate), red (busy)
- Students whose profile includes crowd sensitivity (autism, anxiety) see an automatic banner suggesting quieter routes when their selected route passes through a red zone
- No live sensors needed — entirely time-table-driven prediction updated every hour automatically

**F08 — Sensory Safe Room Finder**

- A dedicated screen listing all quiet rooms and sensory rooms on campus from a pre-seeded database
- Each room entry shows: building name, room number, floor, available equipment (dimmer lights, quiet space, etc.), opening hours, and current status (Available / Occupied / Closed)
- Occupancy is tracked through voluntary digital check-in: student taps "I'm here" on the room page, which increments the occupancy counter in the database
- Status shown as a colour-coded badge: green (Available), amber (Partially Occupied), red (Full)
- "Navigate Here" button on each room card launches the accessible route to that room from the student's current zone
- Accessible from a shortcut on the home screen — one tap, no searching

## Priority 2 — Build and show in demo

**F09 — Timetable-Aware Route Notifications**

- Student enters their weekly class schedule in the app: subject name, building, room number, day, and time
- 15 minutes before each class the app sends a browser push notification: "Time to leave for [Subject] in [Building]. Your accessible route is ready."
- Buffer time in the notification is adjusted by disability type: wheelchair users get 20-minute advance notice, others get 15
- Tapping the notification opens the app directly to the pre-calculated accessible route for that class
- Schedule stored in localStorage — no backend integration needed
- Schedule editable from a simple form in the profile settings section

**F10 — Safe Zone Checkpoints on Routes**

- A pre-seeded dataset of rest spots, benches, covered waiting areas, and medical rooms across campus stored as GeoJSON points
- These appear as blue shield icons on the Leaflet map at all times
- When a route is generated, the panel below the map lists every safe zone checkpoint along the path with its name and distance from the student's current position
- A "Rest-Friendly Route" toggle in the route options switches the algorithm to prioritise paths that pass through the most checkpoints even if slightly longer
- Students can submit a new rest spot via a form — submitted spots go to the admin dashboard for approval before appearing on the live map

**F11 — Gamified Reporting System**

- Every student profile has a points counter and badge collection stored in the database
- Points awarded automatically: submitting a report (5 pts), having a report confirmed by 3 users (15 pts), confirming another user's report (3 pts), completing profile setup (10 pts), using the app 5 days in a row (20 pts)
- Badges: "First Responder" (first report), "Safety Guardian" (10 approved reports), "Community Pillar" (50 confirmations), "Navigator" (20 routes completed)

- A leaderboard page shows top 10 contributors this month — anonymous display names used by default
- Badge collection displayed on the student profile page as a visual grid of earned and locked badges

**F12 — First Aid and Emergency Resource Map**

- All first aid kits, medical rooms, and AED locations pre-seeded in the database as map points
- Shown as red cross icons on the Leaflet map, always visible at all zoom levels
- Clicking any icon opens a popup with: resource name, what is stocked, last inspection date (from seeded data), and a "Navigate Here" button
- Admin can update resource status (Available / Needs Restock / Out of Service) from the admin dashboard
- When SOS is triggered, the three nearest emergency resources are automatically highlighted with a pulsing animation on the student's map

**F13 — High Contrast and Colour-Blind UI Modes**

- A theme switcher in settings with six options: Standard, High Contrast, Deuteranopia, Protanopia, Tritanopia, Full Monochrome
- Each theme is a pre-built CSS class applied to the root `<body>` element — instant switch with no reload
- All map icons use shape and label differentiation in addition to colour so no information is colour-only
- Theme preference saved to localStorage and applied on every app load
- A theme preview strip in settings shows how each theme looks before the student selects it

**F14 — Faculty Accommodation Notification**

- During timetable setup, the student optionally enters the faculty email address for each course
- When the student starts navigation to a class, the app makes an API call to a backend endpoint that sends an automated email via EmailJS (free tier) to the faculty
- Email says: "A differently-abled student is navigating to your [Room] class at [Time]. Please ensure accessible seating is prepared."
- This feature is fully opt-in — a toggle in the profile settings enables or disables it per course
- A log of all sent accommodation notifications is shown in the student's profile under "My Accommodation History"

**F15 — Offline Mode with Cached Map Data**

- On first load, the app registers a service worker that caches: the full campus GeoJSON map, all static route data, the obstacle report list (last known state), safe room list, and emergency resource locations
- When the device goes offline, a banner appears: "You are offline — showing last synced map data"

- All navigation, route calculation, and map rendering continue to work fully offline using cached data
- SOS button in offline mode shows the pre-saved emergency contact numbers in a static display since API calls cannot go out
- When the connection is restored, the service worker syncs any locally queued obstacle reports to the database automatically

## Priority 3 — Unique differentiators that win judges

### F16 — AI Incident Severity Classifier

- When any obstacle report or SOS is submitted, the text description is sent to the Claude API with a classification prompt
- Claude returns a structured JSON: `{ "severity": "critical/serious/minor", "category": "infrastructure/safety/accessibility", "recommended_action": "..." }`
- The severity label is stored with the report and displayed as a colour-coded badge on both the admin dashboard and the student-facing alert
- The recommended action text is shown to the admin as a suggested first-response step below the incident card
- This makes the admin dashboard genuinely AI-powered and not just a CRUD interface — a strong judge impression point

### F17 — Live Campus Announcement Banner

- Admin posts a campus-wide announcement from the dashboard with a title, message, and expiry time
- The announcement appears as a dismissible banner at the top of every student's screen within 30 seconds (via polling every 30 seconds)
- Announcements are colour-coded by type: red for safety alerts, amber for maintenance notices, blue for general information
- The announcement is also read aloud once via SpeechSynthesis for students who have voice mode enabled in their profile
- Students can tap the banner to expand it and see the full message with a timestamp

### F18 — Student Wellbeing Check-In

- A daily check-in prompt appears once per day when the student opens the app: "How are you feeling today? This helps us personalise your route suggestions."
- Student selects from: Energetic, Moderate, Low Energy, Stressed/Anxious
- Based on selection: Low Energy activates rest-friendly routing automatically for the day; Stressed/Anxious activates quiet route mode avoiding busy zones and shows the nearest sensory room
- Check-in takes under 5 seconds and is stored with a date stamp so it never repeats on the same day
- Wellbeing history shown as a simple weekly mood chart on the student profile page using Chart.js

**F19 — Barrier Reporting with AI Auto-Description**

- When a student submits an obstacle report, after they select the issue type and location, they can tap "AI Describe" instead of typing manually
- This sends the issue type and location to Claude API which auto-generates a clear, professional description of the likely issue: e.g. "Elevator in Block C East Wing appears non-functional. This creates a complete access barrier for wheelchair users on floors 2–4."
- Student can edit the auto-generated description before submitting or use it as-is
- This feature removes the burden of writing from students with cognitive or motor impairments who find typing difficult
- The auto-description also ensures all reports in the admin dashboard are consistently worded and actionable

**F20 — Accessible Campus Induction Tour (First-Time Users)**

- On first login, new students are offered an optional 5-step guided tour of the app
- Each step highlights a key feature with an animated tooltip overlay: Profile Setup → Map Navigation → SOS Button → Report an Issue → Safe Rooms
- The tour is narrated via SpeechSynthesis for visually impaired users — each step is read aloud automatically
- Students can skip, pause, or restart the tour at any time from the Help section
- Completing the tour awards 10 bonus points and the "Campus Ready" badge

---

# 3. TECH STACK

**Frontend Framework:** React 18 with Vite — fast build, component-based, ideal for a responsive single-page app

**Styling:** Tailwind CSS — utility-first, fast to build polished UI, no custom CSS files needed

**Map Rendering:** Leaflet.js with React-Leaflet wrapper — free, open-source, no API key required for the map itself

**Map Tile Provider:** OpenStreetMap tiles (free, no key needed) — `https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png`

**Routing Algorithm:** Custom Dijkstra's implementation in plain JavaScript — no external routing library needed, runs client-side on the campus GeoJSON graph

**AI Integration:** Anthropic Claude API (`claude-sonnet-4-20250514`) — for voice assistant, incident classifier, and AI auto-description features

**Speech Input:** Browser Web Speech API (`SpeechRecognition`) — built into Chrome and Edge, no library needed

**Speech Output:** Browser SpeechSynthesis API — built into all modern browsers, no library needed

**Charts:** Chart.js via CDN — for the wellbeing mood chart and admin metrics

**Email Notifications:** EmailJS free tier — for faculty accommodation alerts, no backend email server needed

**Offline Support:** Vite PWA plugin with Workbox — for service worker and asset caching

**Database:** Supabase free tier — PostgreSQL database with a JavaScript client library, handles all reports, users, profiles, announcements, points, and check-ins in real time

**Authentication:** Supabase Auth — email and password login, free tier, handles both student and admin roles

**Notifications:** Browser Notification API — for push alerts and timetable reminders, no third-party service needed

**State Management:** React Context API — no Redux needed, keeps the project simple and fast to build

**Deployment:** Replit's built-in hosting — no configuration needed

---

# 4. PAGES AND USER FLOW

## Public Pages (no login required)

### Page 1 — Landing Page (`/`)

- Full-screen hero with app name "PathAble", tagline "Campus navigation built for every student", and two large buttons: "Get Started" and "Admin Login"
- Below the hero: three feature highlight cards with icons showing the three core value propositions: AI-Guided Routes, Obstacle Alerts, Emergency SOS
- Animated entrance: cards fade in from bottom on scroll using CSS transitions
- "Get Started" navigates to the signup/login page
- No navigation bar on this page — clean and focused

### Page 2 — Login and Signup Page (`/auth`)

- Tab switcher: "Sign In" and "Create Account"
- Sign In form: email, password, Login button
- Create Account form: full name, email, password, role selector (Student / Admin — admin requires a secret code set in environment variable)
- On successful login, if the user has no disability profile saved, redirect to the onboarding flow; otherwise redirect to the home dashboard
- Error messages shown inline below each field

- "Forgot Password" link triggers Supabase password reset email

## Student App Pages (login required, student role)

### Page 3 — Onboarding Flow (`/onboarding`)

- Step 1 of 5: Welcome screen with app mascot illustration and "Let's set up your profile" heading
- Step 2 of 5: "What best describes your situation?" — six large icon cards: Wheelchair User, Visual Impairment, Hearing Impairment, Cognitive/Learning Disability, Temporary Injury, Multiple/Other — multi-select allowed
- Step 3 of 5: "What mobility aid do you use?" — five options: Manual Wheelchair, Electric Wheelchair, Crutches, White Cane, None — single select
- Step 4 of 5: "How would you like navigation instructions?" — three toggle cards: Voice Audio, Haptic Vibration (phone buzz), Visual On-Screen — multi-select
- Step 5 of 5: "Set your emergency contacts" — two fields: Name and Phone Number for up to 2 contacts, plus a toggle for "Notify faculty when I navigate to class"
- Progress bar at the top showing current step
- "Back" and "Next" buttons — "Finish" on the last step saves the profile to Supabase and redirects to the home dashboard
- Completing onboarding awards 10 points and triggers the induction tour offer

### Page 4 — Home Dashboard (`/home`)

- Top bar: PathAble logo on the left, student's first name and avatar initial on the right, notification bell icon
- Daily wellbeing check-in card appears at the top if not yet completed today: "How are you feeling?" with four emoji buttons
- Four large quick-action tiles in a 2x2 grid: Navigate Now, Report an Issue, Find Safe Room, Emergency SOS
- Below the grid: "Active Alerts" section showing any current campus-wide announcements as coloured banners
- Below alerts: "My Points" card showing current points total, level, and the next badge they are working toward
- Bottom navigation bar with five icons: Home, Map, Report, Profile, SOS

### Page 5 — Map and Navigation Screen (`/map`)

- Full-screen Leaflet map taking 60% of the screen height
- Search bar at the top: "Where do you want to go?" with autocomplete suggestions from the campus buildings list
- Route type selector below the search bar: three pill buttons — Shortest, Fewest Obstacles, Most Rest Stops
- "REST-FRIENDLY" toggle switch below the route type selector
- When a destination is selected: the map shows the highlighted route as a green polyline, obstacle pins as red markers, safe zone checkpoints as blue shield icons, and emergency resources as red cross icons

- Turn-by-turn instruction panel below the map: step list with icons for straight, left turn, right turn, and destination
- If voice mode is enabled in profile, each instruction is read aloud via SpeechSynthesis when displayed
- AI Assistant floating button (speech bubble icon) in the bottom right — tapping opens a slide-up chat panel
- AI chat panel: text input at bottom with a microphone icon for voice input, Claude-powered responses shown in chat bubbles, close button at top
- Crowd heatmap toggle button in the map toolbar — tapping overlays colour-coded density zones on the map
- Bottom navigation bar always visible

**Page 6 — Report an Issue (`/report`)**

- Page heading: "Report an Accessibility Issue"
- Step 1: Issue type selector — six large icon buttons: Elevator Down, Wet Floor, Blocked Ramp, Construction, Missing Signage, Other
- Step 2: Zone selector — a searchable dropdown list of all campus buildings and zones
- Step 3: Description field — a text area with placeholder text, plus a large "AI Describe" button below it that auto-fills the description using Claude API based on the selected issue type and zone
- Submit button — on submission, the report is saved to Supabase, the map is updated immediately with the new warning pin, and the student is awarded 5 points
- A confirmation screen appears after submission: "Report submitted. Thank you for keeping campus safe." with the student's updated points total and a "Back to Map" button

**Page 7 — Safe Rooms Finder (`/safe-rooms`)**

- Page heading: "Sensory and Quiet Rooms"
- Filter bar: All, Available, Has Equipment — pill filter buttons
- List of safe room cards, each showing: room name, building, floor, status badge (colour-coded), available equipment as small tags, and opening hours
- Tapping a card expands it to show full details and two buttons: "I'm Here" (check-in) and "Navigate Here" (launches route on the map screen)
- When "I'm Here" is tapped, the occupancy counter increments in Supabase and the badge updates live

**Page 8 — My Schedule (`/schedule`)**

- Page heading: "My Timetable"
- A weekly grid view showing the student's entered classes by day
- "Add Class" button opens a modal form: Subject Name, Building, Room Number, Day (multi-select), Time
- Below each class entry: a small "Faculty Email" field — if filled in, accommodation alerts will be sent to that address automatically when navigation starts

- Toggle switch on each class: "Notify me before this class" — enables the 15-minute advance push notification
- Classes stored in localStorage, editable and deletable

**Page 9 — Student Profile (`/profile`)**

- Avatar circle with student's initials at the top, their name, and email address below
- Points summary: large points number, current badge level label, and a horizontal progress bar to the next level
- Badge grid: 3-column grid of all possible badges — earned badges shown in colour, locked badges shown in grey with a lock icon
- "My Accommodation History" section: list of all faculty accommodation emails sent with date and class name
- Wellbeing mood chart: a 7-day bar chart using Chart.js showing daily mood check-in history
- "Edit Profile" button opens the onboarding flow pre-filled with current data for editing
- Theme switcher section: six theme preview swatches — clicking one applies that theme instantly across the whole app
- "Sign Out" button at the bottom

**Page 10 — Leaderboard (`/leaderboard`)**

- Monthly leaderboard of top 10 point earners across the campus
- Each entry shows: rank number, display name (anonymised by default), points total, and top badge earned
- Student's own rank is highlighted with a different background and "You" label even if not in the top 10 — shown at the bottom of the list
- Toggle: "This Month" / "All Time" switches the leaderboard data
- A "How to earn points" expandable section at the bottom explains all point-earning actions

**Page 11 — Help and Induction Tour (`/help`)**

- Three sections: Restart Tour, Contact Disability Services, Emergency Numbers
- "Restart Tour" button launches the 5-step guided overlay tour on the home dashboard
- Contact section shows disability services office email and phone number from seeded data
- Emergency numbers shows campus security and local emergency services numbers as large tappable links

## Admin Pages (login required, admin role)

**Page 12 — Admin Dashboard (`/admin`)**

- Full-width web layout — not mobile-first like the student app
- Top bar: PathAble Admin, property name, admin's name, sign out

- Four metric cards at the top: Active Incidents, SOS Alerts Today, Reports This Week, Users Online (approximate)
- Three-column main content area:
    - Left column: Active Obstacle Reports — list of all unresolved reports with severity badges and action buttons (Confirm, Dismiss, Escalate)
    - Centre column: Campus Map — same Leaflet map with all pins, read-only, showing real-time incident locations
    - Right column: SOS Alerts and Announcements — SOS event list at top, "Post Announcement" form at the bottom
- "Post Announcement" form: Title field, Message field, Type selector (Safety/Maintenance/General), Expiry time selector — Post button sends to all students instantly

**Page 13 — Admin Resource Manager (`/admin/resources`)**

- Table of all first aid kits, AEDs, and medical rooms with columns: Name, Zone, Floor, Status, Last Updated, Actions
- Status editable inline: Available / Needs Restock / Out of Service — dropdown in the table row
- "Add Resource" button opens a modal form to add a new emergency resource location
- Changes save immediately to Supabase on selection

**Page 14 — Admin Safe Rooms Manager (`/admin/safe-rooms`)**

- Table of all safe rooms with columns: Room Name, Building, Floor, Equipment, Hours, Status, Actions
- Admins can add, edit, and delete safe room entries
- "Pending Submissions" section at the bottom showing safe zone spots submitted by students — Approve or Reject buttons

**Page 15 — Admin Reports Download (`/admin/reports`)**

- Page with three sections: Incident Summary, Accommodation Alert Log, Points and Engagement Summary
- Each section has a date range picker and a "Download as CSV" button
- The CSV is generated client-side from Supabase data using a simple JSON-to-CSV conversion function

---

# 5. UI AND DESIGN INSTRUCTIONS

**Color Scheme:**

- Primary: `#2563EB` (blue) — used for primary buttons, active states, route lines on map

- Secondary: `#10B981` (emerald green) — used for success states, available badges, safe zone icons
- Danger: `#EF4444` (red) — SOS button, critical alerts, emergency resources
- Warning: `#F59E0B` (amber) — serious alerts, crowd heatmap medium density, obstacle pins
- Background: `#F8FAFC` (near white) — main app background
- Surface: `#FFFFFF` — cards, panels, modals
- Text Primary: `#0F172A` (near black)
- Text Secondary: `#64748B` (slate grey)

**Font:** Inter — load from Google Fonts. Use `font-weight: 400` for body text, `font-weight: 600` for headings and labels, `font-weight: 700` for the app name and SOS button only.

**Layout Style:**

- Mobile-first for the student app — max width 430px centred on desktop, with a subtle phone-frame shadow
- Admin dashboard is full-width desktop layout — sidebar or top nav with a three-column content grid
- 16px base padding on all screen edges
- 12px gap between cards in grids, 16px between sections
- All cards use `rounded-2xl` (16px border radius) with a `shadow-sm` and a `1px border` in `#E2E8F0`

**UI Components:**

- Bottom navigation bar: fixed at the bottom of every student screen, 5 icons with labels, active icon filled in primary blue, inactive icons in slate grey
- SOS button: always visible as the rightmost item in the bottom nav — bright red background, white SOS text, slightly larger than other nav items, subtle pulsing ring animation using CSS `@keyframes` to draw attention
- Toast notifications: appear from the top right, auto-dismiss after 4 seconds, colour-coded by type
- Loading states: skeleton loaders (grey animated shimmer blocks) on the map screen and dashboard while data fetches
- Modals: centred overlay with a semi-transparent backdrop, `rounded-2xl`, max width 400px
- The AI chat panel: slides up from the bottom with a smooth CSS transition, handle bar at the top to drag down and close
- Map route line: `#2563EB` blue, 5px stroke weight, with a subtle animated dashed flow using CSS to suggest movement direction
- Obstacle warning pins on map: red circle with exclamation icon
- Safe zone checkpoint pins: blue shield icon
- Emergency resource pins: red cross icon

- Crowd heatmap zones: semi-transparent colour fills over building polygons (green 20% opacity, amber 30% opacity, red 40% opacity)

**Animations:**

- Onboarding step transitions: slide left/right between steps using CSS transform transitions
- Home dashboard cards: fade in from bottom with a 50ms stagger delay between cards on first load
- SOS button: continuous subtle pulse ring using `@keyframes pulse`
- Wellbeing check-in emoji buttons: scale up to 1.2 on hover and tap with a CSS transform
- Badge unlock: a brief confetti burst using a lightweight confetti library (canvas-confetti from CDN) when a new badge is earned
- Map route appearance: the route polyline draws itself progressively using Leaflet's `dashOffset` animation

---

# 6. DATA AND APIs

## Supabase Database Schema

**Table: `users`**

id (uuid, primary key)
email (text)
full_name (text)
role (text: 'student' | 'admin')
created_at (timestamp)

**Table: `profiles`**

id (uuid, primary key)
user_id (uuid, foreign key → users.id)
disability_types (text array)
mobility_aid (text)
nav_modes (text array)
emergency_contact_1_name (text)
emergency_contact_1_phone (text)
emergency_contact_2_name (text)
emergency_contact_2_phone (text)
faculty_notify_enabled (boolean)
theme (text, default: 'standard')
points (integer, default: 0)
badges (text array)

created_at (timestamp)
updated_at (timestamp)

**Table: `obstacle_reports`**

id (uuid, primary key)
reporter_id (uuid, foreign key → users.id)
reporter_name (text)
zone (text)
issue_type (text)
description (text)
ai_severity (text: 'critical' | 'serious' | 'minor')
ai_category (text)
ai_recommended_action (text)
confirmation_count (integer, default: 0)
status (text: 'active' | 'confirmed' | 'dismissed' | 'escalated')
expires_at (timestamp)
created_at (timestamp)

**Table: `sos_alerts`**

id (uuid, primary key)
student_id (uuid, foreign key → users.id)
student_name (text)
disability_types (text array)
zone (text)
status (text: 'active' | 'resolved')
resolved_at (timestamp)
created_at (timestamp)

**Table: `announcements`**

id (uuid, primary key)
admin_id (uuid, foreign key → users.id)
title (text)
message (text)
type (text: 'safety' | 'maintenance' | 'general')
expires_at (timestamp)
created_at (timestamp)

**Table: `safe_rooms`**

id (uuid, primary key)
name (text)

building (text)
floor (integer)
equipment (text array)
opening_hours (text)
status (text: 'available' | 'occupied' | 'closed')
occupancy_count (integer, default: 0)
submitted_by (uuid, nullable)
approved (boolean, default: false for student submissions)

**Table: `emergency_resources`**

id (uuid, primary key)
name (text)
resource_type (text: 'first_aid' | 'aed' | 'medical_room')
building (text)
floor (integer)
zone (text)
status (text: 'available' | 'needs_restock' | 'out_of_service')
last_updated (timestamp)

**Table: `wellbeing_checkins`**

id (uuid, primary key)
user_id (uuid, foreign key → users.id)
mood (text: 'energetic' | 'moderate' | 'low_energy' | 'stressed')
date (date)
created_at (timestamp)

**Table: `accommodation_alerts`**

id (uuid, primary key)
student_id (uuid, foreign key → users.id)
subject_name (text)
room (text)
faculty_email (text)
sent_at (timestamp)

## APIs to Integrate

### Anthropic Claude API

- Endpoint: `POST https://api.anthropic.com/v1/messages`
- Model: `claude-sonnet-4-20250514`

- Headers: `x-api-key: ANTHROPIC_API_KEY`, `anthropic-version: 2023-06-01`, `content-type: application/json`
- Used for: AI assistant responses, incident severity classification, AI auto-description of reports
- Store API key in Replit Secrets as `ANTHROPIC_API_KEY`
- Call from a Vite proxy route (`/api/claude`) to keep the key server-side

**OpenStreetMap Tile API**

- URL template: `https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png`
- No API key required
- Attribution: `© OpenStreetMap contributors`
- Used by Leaflet.js as the base map tile layer

**EmailJS**

- Service URL: `https://api.emailjs.com/api/v1.0/email/send`
- Free tier allows 200 emails per month — sufficient for hackathon
- Create a free account at emailjs.com, set up a Gmail service and a template
- Store `EMAILJS_SERVICE_ID`, `EMAILJS_TEMPLATE_ID`, `EMAILJS_PUBLIC_KEY` in Replit Secrets
- Template variables: `{{faculty_name}}`, `{{student_type}}`, `{{room}}`, `{{time}}`

**Browser Web Speech API (no key needed)**

- `SpeechRecognition` for voice input to the AI assistant
- `SpeechSynthesis` for reading navigation instructions and AI responses aloud
- Both are native browser APIs — no installation or key required

**canvas-confetti (CDN)**

- URL: `https://cdn.jsdelivr.net/npm/canvas-confetti@1.9.2/dist/confetti.browser.min.js`
- Used for the badge-unlock celebration animation
- Call `confetti({ particleCount: 100, spread: 70 })` on badge award

## Mock and Seed Data

Pre-populate the Supabase database with the following seed data by creating a `seed.sql` file and running it in the Supabase SQL editor:

**Campus zones list (used in dropdowns and routing):** Block A (Main Building), Block B (Science Wing), Block C (Arts Block), Block D (Engineering), Library, Student Union, Cafeteria, Sports Complex, Admin Office, Medical Centre, Hostel Block, Parking Zone, Main Gate, East Garden, West Courtyard

**5 pre-seeded safe rooms:**

- Quiet Study Room 1 — Block B, Floor 1, Equipment: Dimmer Lights / Bean Bags / No-phone zone, Hours: 8am–8pm, Status: available
- Sensory Room A — Medical Centre, Floor 0, Equipment: Weighted blankets / Dim lighting / Sound machine, Hours: 9am–5pm, Status: available
- Quiet Library Pod — Library, Floor 2, Equipment: Noise-cancelling headphone station / Dimmable light, Hours: 8am–9pm, Status: available
- Reflection Room — Student Union, Floor 1, Equipment: Prayer mat / Quiet seating / No tech zone, Hours: 7am–10pm, Status: available
- Accessible Study Suite — Block D, Floor 3, Equipment: Height-adjustable desks / Large monitors / Dictation software, Hours: 9am–6pm, Status: available

**8 pre-seeded emergency resources:**

- First Aid Kit 1 — Block A, Floor 0, Ground Floor Lobby
- First Aid Kit 2 — Block C, Floor 1, Near stairwell
- AED Unit 1 — Library, Floor 0, Main entrance wall
- AED Unit 2 — Sports Complex, Floor 0, Reception area
- Medical Room — Medical Centre, Floor 0, Full clinic
- First Aid Kit 3 — Cafeteria, Floor 0, Kitchen entrance
- First Aid Kit 4 — Block D, Floor 2, Corridor junction
- AED Unit 3 — Admin Office, Floor 0, Waiting area

**3 pre-seeded active obstacle reports (for demo):**

- Elevator Down — Block B, Severity: Critical, Created 30 minutes ago
- Wet Floor — Cafeteria, Severity: Minor, Created 1 hour ago
- Blocked Ramp — Main Gate, Severity: Serious, Created 2 hours ago

**Campus timetable density JSON (used for crowd heatmap) — a JavaScript object with this structure:**

```
const densityData = {
  "Block A": { "08": 0.3, "09": 0.9, "10": 0.8, "11": 0.7, "12": 0.4, "13": 0.3, "14": 0.8, "15": 0.7, "16": 0.5, "17": 0.2 },
  "Block B": { "08": 0.2, "09": 0.6, "10": 0.9, "11": 0.8, "12": 0.3, "13": 0.2, "14": 0.7, "15": 0.9, "16": 0.6, "17": 0.2 },
  "Cafeteria": { "08": 0.5, "09": 0.3, "10": 0.2, "11": 0.3, "12": 0.95, "13": 0.9, "14": 0.4, "15": 0.2, "16": 0.2, "17": 0.5 },
  "Library": { "08": 0.4, "09": 0.5, "10": 0.6, "11": 0.7, "12": 0.5, "13": 0.4, "14": 0.6, "15": 0.8, "16": 0.7, "17": 0.5 }
}
```

Values 0–0.4 = green, 0.4–0.7 = amber, 0.7–1.0 = red.

**Campus GeoJSON path graph:** Create a `campus.geojson` file with a `FeatureCollection` of `LineString` features for campus paths and `Point` features for

buildings. Each path feature has properties: `id`, `surface` (paved/gravel/grass), `slope` (flat/mild/steep), `width` (narrow/standard/wide), `has_steps` (boolean). Each building point has properties: `id`, `name`, `zone`, `floors`. Hardcode approximately 30 path segments and 15 building points representing a fictional campus layout. The routing algorithm will use this data to build its graph.

---

# 7. STEP-BY-STEP BUILD INSTRUCTIONS FOR REPLIT AGENT

Follow these instructions in exact order. Do not skip any step. Do not start the next step until the current one is complete and working.

**Step 1 — Project Initialization** Create a new Replit project using the React + Vite template. Open the shell and run: `npm create vite@latest . -- --template react`. Then run `npm install`. Confirm the default Vite app runs at the preview URL before proceeding.

**Step 2 — Install All Dependencies** Run the following single command in the shell: `npm install @supabase/supabase-js react-router-dom leaflet react-leaflet tailwindcss @tailwindcss/vite autoprefixer @headlessui/react lucide-react chart.js react-chartjs-2 emailjs-com vite-plugin-pwa workbox-window`

Then run `npx tailwindcss init -p` to generate the Tailwind config file.

**Step 3 — Configure Tailwind CSS** Open `tailwind.config.js` and set the `content` array to: `["./index.html", "./src/**/*.{js,jsx,ts,tsx}"]`. Open `src/index.css` and replace all content with the three Tailwind directives: `@tailwind base;`, `@tailwind components;`, `@tailwind utilities;`. Add the Inter font import from Google Fonts as the first line of `index.css`: `@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700&display=swap');`. Set the base font in `index.css` body: `font-family: 'Inter', sans-serif;`.

**Step 4 — Configure Vite** Open `vite.config.js`. Add the VitePWA plugin import and configuration for offline support. Add a server proxy configuration so that requests to `/api/claude` are handled by a local Express endpoint (see Step 6). The proxy should forward to `http://localhost:3001/api/claude`.

**Step 5 — Set Up Environment Variables** In Replit Secrets, add the following keys: `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `ANTHROPIC_API_KEY`, `VITE_EMAILJS_SERVICE_ID`, `VITE_EMAILJS_TEMPLATE_ID`,

VITE_EMAILJS_PUBLIC_KEY. Create a .env file in the project root referencing these secrets using import.meta.env.VITE_* naming convention for client-side variables.

**Step 6 — Create the Express API Server for Claude** Install Express: npm install express cors. Create a file server.js in the project root. This file runs a simple Express server on port 3001 with a single POST endpoint /api/claude. This endpoint reads the request body containing { messages, systemPrompt }, makes a fetch call to https://api.anthropic.com/v1/messages with the ANTHROPIC_API_KEY from process.env, and returns the Claude response to the client. Add CORS middleware to allow requests from the Vite dev server. Add a start script to package.json: "server": "node server.js". Run both Vite and the server concurrently by installing concurrently and updating the dev script to: "concurrently \"vite\" \"node server.js\"".

**Step 7 — Set Up Supabase** Create a src/lib/supabase.js file that initialises the Supabase client using VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY. Export the client as the default export. In the Supabase dashboard, run the full schema SQL to create all 9 tables defined in Section 6 with all columns, data types, and foreign key relationships. Enable Row Level Security on all tables. Create policies: authenticated users can read all reports and announcements; users can only insert and update their own profile and reports; only admin role users can update resource and safe room status; only admin role users can insert announcements. Run the seed SQL to populate the pre-seeded data.

**Step 8 — Set Up React Router** Open src/main.jsx. Wrap the App component with <BrowserRouter> from react-router-dom. Open src/App.jsx. Import Routes and Route from react-router-dom. Create route entries for all 15 pages listed in Section 4. Create placeholder component files for each page in src/pages/ directory now so routing works even before each page is fully built. Create a ProtectedRoute wrapper component that checks Supabase auth state and redirects to /auth if not logged in. Create an AdminRoute wrapper that additionally checks the user's role is 'admin' and redirects to /home if not.

**Step 9 — Build the Authentication System** Create src/contexts/AuthContext.jsx with a React Context that stores the current user object and profile. The context should: on mount, call supabase.auth.getSession() to restore any existing session; subscribe to supabase.auth.onAuthStateChange to keep the user state updated; expose signIn, signUp, signOut functions; expose the user, profile, and loading states. Build the /auth page with a tab switcher for Sign In and Create Account forms. The Create Account form should call Supabase auth signUp, then insert a new row into the users table with the returned user ID and selected role. After successful login, the app checks if the user has a row in the profiles table — if not, redirect to /onboarding; if yes, redirect to /home.

**Step 10 — Build the Onboarding Flow** Create src/pages/Onboarding.jsx. Manage a local state object profileData with all profile fields. Render the correct step content based

on a `currentStep` integer state (1 through 5). Each step should use large, tappable card buttons for selections. The progress bar at the top should be a simple div with a width percentage calculated from the current step. On the final step's "Finish" button click, call `supabase.from('profiles').insert(profileData)` with the complete profile object. Award 10 points by updating the points field. Then navigate to `/home`. If the user already has a profile and visits this page, pre-fill all fields from their existing profile and use `upsert` instead of `insert` on finish.

**Step 11 — Build the Home Dashboard** Create `src/pages/Home.jsx`. On mount, fetch: the current user's profile from `profiles`, any active announcements from `announcements` where `expires_at` is in the future, and today's wellbeing check-in from `wellbeing_checkins` where `date` equals today's date. Render the wellbeing check-in card if no check-in exists today. When a mood emoji is tapped, insert a row into `wellbeing_checkins` and update the student's daily routing mode accordingly (low energy → set a `restFriendly` flag in context; stressed → set a `quietMode` flag). Render the four quick-action tiles as large rounded cards with icons from `lucide-react`. Render the announcements as coloured banners using the type field to select the colour. Render the points card using data from the profile. Build the `BottomNav` component and import it here — it will be used on all student pages.

**Step 12 — Build the Campus Map and Navigation Screen** Create `src/pages/MapNav.jsx`. This is the most complex page — build it carefully. Import `MapContainer`, `TileLayer`, `Polyline`, `Marker`, `Popup`, `Circle` from `react-leaflet`. Import `L` from `leaflet`. Create custom Leaflet icon objects for obstacle pins (red), safe zone pins (blue shield), and emergency resource pins (red cross) using `L.divIcon` with inline SVG strings. On mount, load the campus GeoJSON from `src/data/campus.geojson`. Build the graph data structure: an adjacency list where each path segment is an edge weighted by its distance and penalised by slope and width based on the student's disability profile. Implement Dijkstra's algorithm as a pure JavaScript function in `src/utils/routing.js` that takes the graph, start node, and end node and returns the shortest path as an array of coordinate pairs. When the student selects a destination from the search bar autocomplete, run the routing function and store the resulting coordinate array in state — the Leaflet `Polyline` component renders it on the map automatically. Fetch active obstacle reports from Supabase on mount and every 60 seconds. Render each active report as a red marker on the map. Fetch emergency resources and safe zone checkpoints and render their markers. Build the turn-by-turn instruction panel by converting the route coordinate array into human-readable instructions: calculate the bearing change between each consecutive pair of coordinates to determine turn direction. If the student's profile has voice mode enabled, use `window.speechSynthesis.speak()` to read each instruction as it appears. Build the AI chat panel as a slide-up div. Wire the microphone button to the Web Speech API `SpeechRecognition` object. When recognition returns a transcript, send it to `/api/claude` with a system prompt containing the campus knowledge base as a JSON string. Display the response in the chat panel and speak it via SpeechSynthesis. Build the crowd density heatmap toggle: when toggled on, import the

density JSON, get the current hour from `new Date().getHours()`, and render `Circle` components on the map for each zone at its density-based colour.

**Step 13 — Build the Report an Issue Page** Create `src/pages/Report.jsx`. Step 1 renders six icon button cards for issue type. Step 2 renders a searchable select dropdown with all campus zone names. Step 3 renders a textarea and the "AI Describe" button. Wiring the "AI Describe" button: send the selected `issueType` and `zone` as a user message to `/api/claude` with a system prompt instructing Claude to write a concise, professional accessibility incident description. Place the returned text into the textarea. On submit: insert the report into `obstacle_reports`, call `/api/claude` again with a classification prompt asking for severity, category, and recommended action as JSON, save the AI response fields into the report row via an update call, award 5 points to the student by incrementing the `points` field in their profile, navigate to a confirmation screen.

**Step 14 — Build the Safe Rooms Finder Page** Create `src/pages/SafeRooms.jsx`. Fetch all approved safe rooms from Supabase on mount. Render filter pill buttons that filter the local state array. Render room cards using a `map` over the filtered rooms. Each card uses a `useState` for expanded/collapsed state. The "I'm Here" button calls `supabase.from('safe_rooms').update({ occupancy_count: room.occupancy_count + 1 })` and updates the local status badge. The "Navigate Here" button saves the selected room's zone to a React Context and navigates to `/map` — the map screen reads this context on mount and pre-fills the destination.

**Step 15 — Build the Schedule, Profile, Leaderboard, and Help Pages** Create `src/pages/Schedule.jsx`: read and write schedule data to localStorage using a custom hook `useLocalStorage`. Render a weekly grid using CSS grid with 7 columns. The "Add Class" button opens a headlessUI `Dialog` modal with the class form fields. Each class card shows a "Notify me" toggle and an expandable row for the faculty email field. Create `src/pages/Profile.jsx`: fetch the profile from Supabase. Render the points number and progress bar. Build the badge grid with all 10 possible badges — filter by the profile's `badges` array to determine earned vs locked. Render the wellbeing chart using `react-chartjs-2 Bar` chart fed the last 7 days of `wellbeing_checkins` data. Build the theme switcher as six coloured swatch buttons that add a class to `document.body` and save the selection to localStorage. Create `src/pages/Leaderboard.jsx`: query Supabase for all profiles ordered by points descending, take the top 10. Render the list. Find and highlight the current user's rank. Create `src/pages/Help.jsx`: a static page with three sections as described. The "Restart Tour" button sets a `tourActive` state in a global context.

**Step 16 — Build the Admin Dashboard** Create `src/pages/admin/Dashboard.jsx`. This page uses a full-width three-column grid layout. Left column: fetch all `obstacle_reports` where status is not 'dismissed'. Map over them to render incident cards. Each card has three buttons: Confirm (updates status to 'confirmed' and increments `confirmation_count`), Dismiss (updates status to 'dismissed'), Escalate (updates status to 'escalated' and sends the admin an in-app notification). Use Supabase's `realtime`

subscription on the `obstacle_reports` table so new reports appear without a page refresh. Centre column: the same Leaflet map as the student map but read-only and full-height, showing all active incident pins. Right column top: list of SOS alerts from the `sos_alerts` table filtered to active status. Right column bottom: the Post Announcement form — on submit, insert into `announcements` with the expiry time calculated by adding the selected hours to `new Date()`. Build `src/pages/admin/Resources.jsx` and `src/pages/admin/SafeRooms.jsx` as table-based CRUD pages using Supabase for all operations. Build `src/pages/admin/Reports.jsx` with the three downloadable sections — implement a `jsonToCSV` utility function that converts a Supabase query result array into a comma-separated string and triggers a browser download via a temporary anchor element with `href: 'data:text/csv;charset=utf-8,' + encodeURIComponent(csvString)`.

**Step 17 — Implement the Gamification Logic** Create `src/utils/gamification.js` with a function `awardPoints(userId, points, badgeToCheck)`. This function: increments the user's points by the given amount using a Supabase RPC function or an update query, checks if the new total crosses any badge thresholds or if specific actions trigger badges, if a badge is newly earned adds it to the `badges` array in the profile and triggers `confetti()` from the canvas-confetti library, returns the updated profile. Call this function from: Report submission (5 pts), Report confirmation (15 pts or 3 pts), Profile completion (10 pts), Tour completion (10 pts), Daily streak logic checked on each app open.

**Step 18 — Implement Offline Mode** Configure the VitePWA plugin in `vite.config.js` to cache: all static assets, the campus GeoJSON file, and the app shell. Create a service worker strategy that serves cached responses when offline. In `src/App.jsx`, use the `useRegisterSW` hook from `workbox-window` to register the service worker. Add an online/offline event listener to `window` that sets a global `isOnline` state. Show the offline banner when `isOnline` is false. When offline and the student submits an obstacle report, save it to a `pendingReports` array in localStorage. When the app comes back online, send all pending reports to Supabase and clear the localStorage queue.

**Step 19 — Implement the Induction Tour** Create `src/components/InductionTour.jsx`. This component renders a series of overlay tooltips pointing to specific elements on the home dashboard using `getBoundingClientRect()` to position the tooltip box relative to the target element. Store the current tour step in a `tourStep` state (0 = inactive, 1–5 = active steps). Each step highlights one element, shows a description, and has "Next" and "Skip" buttons. At step 5 completion, award 10 points and the "Campus Ready" badge. Trigger speech synthesis to read each step description when voice mode is active. Import and render `InductionTour` in `src/App.jsx` so it overlays any page. Control its visibility from global context.

**Step 20 — Final Polish and Testing** Add the SOS pulsing animation to the bottom nav SOS icon using this CSS in `index.css`: `@keyframes sos-pulse { 0%, 100% { box-shadow: 0 0 0 0 rgba(239,68,68,0.4); } 50% { box-shadow: 0 0 0`

`8px rgba(239,68,68,0); } }` and apply it to the SOS button element. Test the full user flow: new user signup → onboarding → home dashboard → navigate to a location → report an obstacle → view safe rooms → trigger SOS → check leaderboard. Test the admin flow: login as admin → view dashboard → post an announcement → confirm a report → update a resource status. Verify the announcement appears on the student screen within 30 seconds. Verify that the AI assistant responds to "Where is the library?" with a sensible answer. Verify the crowd heatmap toggles on and off correctly. Check all pages render correctly on a 390px wide mobile viewport. Run `npm run build` and confirm there are no build errors. Verify the app is accessible at the Replit preview URL and all routes work with direct URL access.

---

**Notes for Replit Agent:**

- Use `lucide-react` for all icons throughout the app — do not use any other icon library
- All API keys must be read from environment variables — never hardcode them in source files
- Use Tailwind utility classes exclusively for styling — do not write custom CSS except for the animations and font import in `index.css`
- All Supabase queries must include error handling — log errors to the console and show a user-friendly toast notification on failure
- The campus GeoJSON in `src/data/campus.geojson` must be a valid GeoJSON FeatureCollection — validate it with a GeoJSON linter before using it
- The routing algorithm must handle the case where no accessible path exists between two points and show a "No accessible route found" message rather than crashing
- The app must work on both Chrome and Safari mobile browsers — test the Web Speech API fallback for Safari where `SpeechRecognition` may not be available (show a text input instead)